

Aerolytics: Technical Study Guide (Backend & Causality)

This guide breaks down the complex technical features of the backend and the causality engine into simple, easy-to-study language for your viva exam.

Part 1: The Backend Engine

Think of the backend as the brain of the application. It runs 24/7, fetching data, running math models, and serving it to the frontend screen.

1. The Framework: FastAPI

- **What we used:** Python with a framework called **FastAPI**.
- **Why we used it:** Most older frameworks (like Django or Flask) process one task at a time. FastAPI is "asynchronous." This means it can juggle multiple tasks at once—like fetching weather data, running an AI prediction, and answering a chatbot question—without freezing or slowing down.

2. Data Ingestion: Background Workers

- **The Problem:** If the app asks the government servers (OpenAQ) for data every single time a user opens the dashboard, the app would be incredibly slow and might crash.
- **Our Solution:** We use a tool called **APScheduler**. We created "background workers" that wake up automatically every hour. They quietly fetch all the live pollution and weather data for 23 cities and save it directly onto our own server in **Local CSV Buffers**.
- **The Result:** When a user opens the app, the data loads instantly in milliseconds because it's reading from our local buffer, not waiting on a slow external server.

3. Consistency: The Rankings Cache

- **The Concept:** We have a Map page and a Dashboard page. We need to make sure they both show the exact same AQI number at all times.
 - **How it works:** The backend maintains a rankings.json file. A background task recalculates the national rankings every 5 minutes and updates this file. This acts as our **"Single Source of Truth."** Whenever the frontend needs data, it reads this one file, ensuring 100% consistency across the whole app.
-

Part 2: The Causality Module (Our Major Novelty)

This is the most unique part of the project. Most AQI apps tell you *what* the pollution level is. Our Causality Module tells you *why* it is happening.

1. The Flaw in Old Apps: Correlation

- Standard apps use "Correlation." This just means two things happen at the same time.
- *Example:* Ice cream sales and sunburns both go up in the summer. They are correlated. But eating ice cream does NOT *cause* sunburns.

- In pollution, just because the temperature drops and PM2.5 goes up, it doesn't mean the temperature directly caused it. Correlation is often a coincidence.

2. Our Solution: Granger Causality

To prove what is actually driving the pollution, we built a math engine (app/services/causal.py) that uses a strict statistical test called **Granger Causality**.

We use a mathematical method called **Vector Autoregression (VAR)**.

3. How Granger Causality Works (In Simple Terms)

1. **The Target:** We take our main pollutant, like PM2.5 (fine dust).
2. **The Suspect:** We take a suspected cause, like NO2 (traffic fumes) or Wind Speed.
3. **The Test:** The engine looks back in time (we call these "lags", testing up to 6 hours into the past). It asks the math equation: *"If I include the history of traffic fumes, does my prediction of PM2.5 dust get significantly better?"*
4. **The Proof:** If the answer is yes, then the traffic fumes contain unique information that drives the dust. We have proven a directional, causal link.

4. Making the Math Human-Readable

The result of this heavy math is a "P-value" (like $P < 10^{-20}$). Regular users can't understand this.

- Our backend has a translator built in.
- If the P-value is incredibly tiny, the backend turns it into the phrase **"Extremely Strong Link."**
- It then pairs this with a pre-written English explanation, so the frontend displays a simple card saying: *"Extremely Strong Link: Traffic emissions (NO2) are a key contributor."*

Summary for the Examiner

If asked what makes this project technically impressive:

"We built an asynchronous FastAPI backend that uses background workers to ensure zero-latency data loading. But more importantly, we moved past basic correlation. We implemented Granger Causality using Vector Autoregression to mathematically prove the directional drivers of pollution, turning complex statistical P-values into actionable, plain-English insights for the user."